Descriptive Statistics & Python Data Libraries

NumPy, Pandas, and the Mathematics of Data Summarization

Course: Data Analysis & AI Fundamentals
Instructor: Victor Wandera Lumumba, MSc.

Contents

1 The Role of Descriptive Statistics in Data Science

Before we can build predictive machine learning models or deploy artificial intelligence systems, we must first understand the data we are working with. This is the domain of **Descriptive Statistics**.

Definition: Descriptive vs. Inferential Statistics

Descriptive Statistics summarize and organize the features of a dataset in a way that allows for quick interpretation. It describes *what is happening in the data right now.* **Inferential Statistics**, on the other hand, uses a sample of data to make inferences, predictions, or generalizations about a larger population (e.g., hypothesis testing, A/B testing, regression analysis).

In the Data Analysis Lifecycle, descriptive statistics form the backbone of **Exploratory Data Analysis (EDA)**. When you receive a raw dataset—say, clinical records for a CardioPredict AI model or financial transactions for a fraud detection system—you cannot immediately feed it into an algorithm. You must first profile the data: What is the average patient age? How much does the blood pressure vary? Are there extreme outliers? Are the data distributions skewed? Answering these questions prevents the "Garbage In, Garbage Out" (GIGO) phenomenon and guides your data cleaning and feature engineering strategies.

2 Measures of Central Tendency

Measures of central tendency attempt to identify a single value that represents the "center" or "typical" value of a dataset. They provide a single summary figure that is highly useful for stakeholder reporting.

2.1 The Arithmetic Mean

The mean is the most common measure of central tendency. It is calculated by summing all observations and dividing by the total number of observations.

Mathematical Foundation: Population vs. Sample Mean

For a **population** of size N, the population mean (μ) is:

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i \tag{1}$$

For a **sample** of size n drawn from that population, the sample mean ($\bar{x}$) is:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \tag{2}$$

Properties of the Mean:

- It uses every single data point in its calculation.
- It is the **balance point** of the data; the sum of the deviations from the mean is always zero.
- **Major Weakness:** It is highly sensitive to **outliers**. In a dataset of annual incomes where 99 people earn \$50,000 and 1 person earns \$10,000,000, the mean income will be

heavily skewed and fail to represent the "typical" person.

2.2 The Median

The median is the exact middle value of a dataset when it is ordered from smallest to largest.

- If n is odd, the median is the value at position $\frac{n+1}{2}$.
- If n is even, the median is the average of the two middle values.

Properties of the Median:

- It is a **robust statistic**. It is completely unaffected by extreme outliers. In the income example above, the median remains \$50,000, which is a much better representation of the typical citizen.
- It is the preferred measure of central tendency for **skewed distributions** (e.g., survival times in medical studies, which ties directly into survival analysis techniques like the Kaplan-Meier estimator).
- It can be calculated for ordinal data (e.g., Likert scales), whereas the mean cannot.

2.3 The Mode

The mode is the value that appears most frequently in a dataset.

- A dataset can be **unimodal** (one mode), **bimodal** (two modes), or **multimodal**.
- It is the **only** measure of central tendency that can be used for **nominal categorical data** (e.g., the most common blood type in a hospital dataset, or the most frequent diagnosis code).

2.4 The Impact of Skewness

The relationship between the mean, median, and mode reveals the **skewness** of a distribution:

- **Symmetric Distribution:** Mean $\approx$ Median $\approx$ Mode (e.g., Normal distribution of human heights).
- **Right-Skewed (Positive Skew):** Mean $>$ Median $>$ Mode. The tail extends to the right (e.g., income, insurance claims).
- **Left-Skewed (Negative Skew):** Mean $<$ Median $<$ Mode. The tail extends to the left (e.g., age at death in developed nations).

3 Measures of Dispersion (Variability)

Knowing the center of the data is only half the story. Two datasets can have the exact same mean but vastly different spreads. In finance, spread represents **risk** (volatility). In healthcare, spread represents **heterogeneity** in patient responses.

3.1 Range and Interquartile Range (IQR)

- **Range:** Maximum − Minimum. Highly sensitive to outliers; rarely used in rigorous statistical analysis on its own.
- **Quartiles:** Values that divide a sorted dataset into four equal parts. Q1 (25th percentile), Q2 (50th percentile / Median), Q3 (75th percentile).

- **Interquartile Range (IQR):** The range of the middle 50% of the data.

$$IQR = Q3 - Q1 \tag{3}$$

The IQR is a robust measure of spread. It is the foundational metric for detecting outliers using the **1.5 × IQR rule**: any point below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ is considered a mild outlier.

3.2 Variance and Standard Deviation

Variance and standard deviation measure the average distance of each data point from the mean.

Mathematical Foundation: Variance and Standard Deviation Formulas

Population Variance (σ^2):

$$\sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2 \tag{4}$$

Sample Variance (s^2):

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2 \tag{5}$$

Standard Deviation is simply the square root of the variance: $s = \sqrt{s^2}$.

Under the Hood: Why divide by $n-1$? (Bessel's Correction)

You might wonder why the sample variance divides by $n-1$ instead of n. This is known as **Bessel's correction**. When we calculate the sample variance, we use the *sample mean* ($\bar{x}$) instead of the true *population mean* (μ). Because the data points are, on average, closer to their own sample mean than to the true population mean, dividing by n would systematically *underestimate* the true population variance. Dividing by $n-1$ corrects this bias, making s^2 an **unbiased estimator** of σ^2.

Why Standard Deviation? Variance is expressed in squared units (e.g., "dollars squared" or "beats-per-minute squared"), which is cognitively difficult to interpret. Taking the square root returns the metric to the original units of the data, making the Standard Deviation the industry standard for reporting variability.

3.3 Coefficient of Variation (CV)

When comparing the variability of two datasets with completely different units or vastly different means (e.g., comparing the volatility of a \$10 stock vs. a \$1000 stock), standard deviation is misleading. The **Coefficient of Variation** standardizes the dispersion:

$$CV = \left(\frac{\text{Standard Deviation}}{\text{Mean}}\right) \times 100\% \tag{6}$$

It expresses the standard deviation as a percentage of the mean, allowing for apples-to-apples comparisons of relative risk or variability.

4 Measures of Shape and Relationship

4.1 Skewness and Kurtosis

Beyond central tendency and spread, we must understand the **shape** of the distribution.

- **Skewness:** Measures the asymmetry of the distribution. A skewness of 0 is perfectly symmetric (like the Normal distribution). Positive skew means a long right tail; negative skew means a long left tail. Many machine learning algorithms (like Linear Regression) assume normally distributed residuals, making skewness a critical metric to check before modeling.
- **Kurtosis:** Measures the "tailedness" of the distribution. High kurtosis (leptokurtic) means heavy tails and a sharp peak, indicating a higher probability of extreme outliers. Low kurtosis (platykurtic) means light tails and a flat peak.

4.2 Covariance and Correlation

While the previous metrics analyze single variables (univariate analysis), we often need to understand how two variables move together (bivariate analysis).

Covariance indicates the *direction* of the linear relationship between two variables.

$$\text{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) \tag{7}$$

However, covariance is unstandardized. A value of 500 might indicate a strong relationship or a weak one, depending on the scales of X and Y.

To fix this, we use the **Pearson Correlation Coefficient** (r), which standardizes covariance by dividing it by the product of the standard deviations of X and Y:

$$r = \frac{\text{Cov}(X, Y)}{s_x s_y} \tag{8}$$

The correlation coefficient r is strictly bounded between -1 and 1.

- $r = 1$: Perfect positive linear relationship.
- $r = -1$: Perfect negative linear relationship.
- $r = 0$: No *linear* relationship.

Common Pitfall: Correlation $\neq$ Causation

A high correlation between two variables does not mean one causes the other. They might both be driven by a hidden third variable (a confounding variable), or the correlation might be entirely coincidental (spurious correlation). Furthermore, Pearson's r only captures *linear* relationships. A perfect non-linear relationship (like a parabola) might yield an r value close to 0. Always visualize your data using scatter plots!

5 The Limitation of Summary Statistics: Anscombe's Quartet

Before moving to Python, it is vital to understand a statistical trap known as **Anscombe's Quartet**. Coined by statistician Francis Anscombe in 1973, it consists of four distinct datasets that have nearly identical descriptive statistics:

- Same mean for X and Y.
- Same variance for X and Y.

- Same correlation between X and Y.
- Same linear regression line.

However, when **graphed**, the four datasets look completely different: one is a linear relationship, one is a non-linear curve, one is a perfect line with a single outlier, and one is a vertical line with an outlier. **The Lesson:** Descriptive statistics are incredibly powerful, but they can be blind to the underlying structure of the data. They must *always* be paired with data visualization (histograms, scatter plots, boxplots).

6 Introduction to NumPy: The Engine of Python Data Science

To compute these statistics programmatically, we turn to Python. While Python's built-in 'math' module is fine for basic arithmetic, it is entirely inadequate for data science. We need **NumPy** (Numerical Python).

6.1 Why Not Standard Python Lists?

Python lists are arrays of pointers to objects scattered across your computer's memory. If you want to add two lists of 1 million numbers together, Python has to check the type of every single object, dereference the pointers, and perform the addition one by one. This is incredibly slow.

Under the Hood: NumPy's Secret Weapon: Contiguous Memory & Vectorization

NumPy introduces the 'ndarray' (N-dimensional array). Unlike Python lists, a NumPy array stores data of the **exact same type** (e.g., all 64-bit floats) in **contiguous blocks of memory**. Because the data is contiguous and uniformly typed, NumPy can leverage **SIMD** (Single Instruction, Multiple Data) vectorized CPU instructions. It performs operations on the entire array in optimized C-code without ever entering the slow Python interpreter loop. This makes NumPy up to 100x faster than standard Python loops.

6.2 Core NumPy Concepts

```
import numpy as np

# Creating arrays
arr_1d = np.array([10, 20, 30, 40])
arr_2d = np.array([[1, 2, 3], [4, 5, 6]]) # Matrix

# Array attributes
print(arr_2d.shape)  # (2, 3) -> 2 rows, 3 columns
print(arr_2d.dtype)  # int64 (or int32 depending on OS)
print(arr_2d.ndim)   # 2 (Dimensions)
```

6.3 Vectorized Statistical Operations

NumPy provides highly optimized functions for the descriptive statistics we discussed.

```
# Simulating patient recovery times (in days)
recovery_times = np.array([5, 7, 6, 8, 5, 10, 45, 6, 7, 5])
# Note: 45 is a massive outlier (perhaps a data entry error)

print("Mean:", np.mean(recovery_times))       # Skewed by 45
```

```
print("Median:", np.median(recovery_times))   # Robust to 45
print("Std Dev:", np.std(recovery_times, ddof=1)) # ddof=1 for sample
    std dev
print("IQR:", np.percentile(recovery_times, 75) - np.percentile(
    recovery_times, 25))
```

Pro Tip: The 'ddof' Parameter

By default, 'np.std()' and 'np.var()' calculate the *population* standard deviation (dividing by N). In data science, we almost always work with *samples*. To get the sample standard deviation (dividing by $N-1$, applying Bessel's correction), you must set the Delta Degrees of Freedom parameter: 'ddof=1'.

6.4 Broadcasting

Broadcasting is NumPy's ability to perform arithmetic operations on arrays of **different shapes**. It allows you to write concise code without explicitly writing loops.

```
# Standardizing data (Z-score normalization)
# Formula: z = (x - mean) / std
data = np.array([10, 20, 30, 40, 50])

mean_val = np.mean(data)
std_val = np.std(data, ddof=1)

# Broadcasting automatically applies the scalar to every element
z_scores = (data - mean_val) / std_val
print(z_scores) # [-1.414, -0.707, 0.0, 0.707, 1.414]
```

7 Introduction to Pandas: The Data Wrangling Workhorse

While NumPy is the mathematical engine, **Pandas** is the interface designed specifically for handling structured, tabular data (like SQL tables or Excel spreadsheets). Built on top of NumPy, Pandas introduces two vital data structures: the **Series** (1D) and the **DataFrame** (2D).

7.1 The DataFrame and Series

A **Series** is a single column of data with an associated **Index** (labels for each row). A **DataFrame** is essentially a dictionary of Series objects that all share the same index.

```
import pandas as pd

# Creating a DataFrame from a dictionary
data = {
    'patient_id': ['P001', 'P002', 'P003', 'P004'],
    'age': [45, 34, 55, 29],
    'blood_pressure': [120, 135, 140, 118],
    'diagnosis': ['Hypertension', 'Normal', 'Hypertension', 'Normal']
}

df = pd.DataFrame(data)
print(df)
```

7.2 Loading and Inspecting Real-World Data

In the real world, data comes from external files. Pandas supports dozens of formats, with CSV and Excel being the most common.

```
# Loading data
# df = pd.read_csv('mediacrest_students.csv')
# df = pd.read_excel('financial_data.xlsx', sheet_name='Q1')

# Initial Inspection (The First 5 Commands You Should Always Run)
df.head()       # View first 5 rows
df.tail()       # View last 5 rows
df.shape        # Returns a tuple: (number_of_rows, number_of_columns)
df.dtypes       # Shows the data type of each column (crucial for
    memory management)
df.info()       # Comprehensive summary: index dtype, columns, non-null
     values, memory usage
```

7.3 The Magic of '.describe()'

The '.describe()' method is the bridge between Pandas and Descriptive Statistics. It automatically computes the core summary metrics for the dataset.

```
# For numerical columns, it outputs:
# count, mean, std, min, 25%, 50% (median), 75%, max
print(df.describe())

# For categorical columns, you must explicitly ask for them:
# It outputs: count, unique, top (mode), freq
print(df.describe(include=['object', 'category']))
```

7.4 Data Selection and Filtering

Pandas provides powerful ways to slice and dice data. The two primary indexers are '.loc' (label-based) and '.iloc' (integer-position-based).

```
# Selecting a single column (Returns a Series)
ages = df['age']

# Selecting multiple columns (Returns a DataFrame)
subset = df[['age', 'blood_pressure']]

# Filtering rows based on conditions (Boolean Indexing)
high_risk = df[df['blood_pressure'] > 130]

# Combining multiple conditions (MUST use parentheses & bitwise
    operators)
target_group = df[(df['age'] > 40) & (df['diagnosis'] == 'Hypertension'
    )]

# Using .loc for simultaneous row and column filtering
# Syntax: .loc[row_condition, column_labels]
result = df.loc[df['age'] > 40, ['patient_id', 'blood_pressure']]
```

7.5 Handling Missing Data

Missing data is represented in Pandas as 'NaN' (Not a Number), which is a special floating-point value defined by the IEEE 754 standard. Because it is a float, the presence of even a single 'NaN' in an integer column will force Pandas to upcast the entire column to 'float64'.

```
# Identifying missing data
missing_counts = df.isna().sum()
missing_percent = (df.isna().sum() / len(df)) * 100

# Strategy 1: Dropping missing data
# Drop rows where ANY value is missing
df_dropped = df.dropna()
# Drop columns where > 50% of values are missing
df_dropped_cols = df.dropna(thresh=len(df)*0.5, axis=1)

# Strategy 2: Imputation (Filling missing data)
# Fill missing ages with the median (Robust to outliers)
df['age'] = df['age'].fillna(df['age'].median())

# Forward fill or backward fill (Useful for time-series data)
df['blood_pressure'] = df['blood_pressure'].ffill()
```

7.6 Aggregation and the 'groupby' Paradigm

The 'groupby' method implements the **Split-Apply-Combine** strategy, which is fundamental to data analysis. You split the data into groups based on some criteria, apply a function (like mean or sum) to each group independently, and combine the results into a new data structure.

```
# Split-Apply-Combine: Average blood pressure by diagnosis
grouped_means = df.groupby('diagnosis')['blood_pressure'].mean()

# Applying multiple aggregation functions at once
summary_stats = df.groupby('diagnosis').agg({
    'age': ['mean', 'median', 'std'],
    'blood_pressure': ['min', 'max', 'count']
})

# Resetting the index to turn the result back into a flat DataFrame
summary_flat = summary_stats.reset_index()
```

8 Advanced Pandas Techniques for Efficiency

8.1 Method Chaining

To write clean, readable, and reproducible data cleaning pipelines, Pandas supports **method chaining**. Instead of creating intermediate variables, you chain methods together using parentheses.

```
# Without method chaining (Cluttered, creates temporary variables)
df_temp1 = df.dropna(subset=['age'])
df_temp2 = df_temp1[df_temp1['age'] > 18]
df_final = df_temp2.assign(age_group=lambda x: np.where(x['age'] > 50,
    'Senior', 'Adult'))

# With method chaining (Clean, readable, functional style)
df_final = (
```

```
    df
    .dropna(subset=['age'])
    .loc[lambda x: x['age'] > 18]
    .assign(age_group=lambda x: np.where(x['age'] > 50, 'Senior', '
    Adult'))
)
```

8.2 Vectorized String Operations

When dealing with messy categorical data (e.g., "Nairobi", "nairobi ", "NAIROBI"), do not use Python 'for' loops or '.apply()' with custom functions. Pandas provides vectorized string methods via the '.str' accessor, which operates at C-speed.

```
# Messy city names
cities = pd.Series(['Nairobi', ' nairobi ', 'MOMBASA', 'Kisumu'])

# Vectorized cleaning
clean_cities = (
    cities
    .str.strip()          # Remove leading/trailing whitespace
    .str.lower()          # Convert to lowercase
    .str.capitalize()     # Capitalize first letter
)
print(clean_cities)
# Output: ['Nairobi', 'Nairobi', 'Mombasa', 'Kisumu']
```

9 Summary and Best Practices

Mastering descriptive statistics and the Python data stack (NumPy and Pandas) is the non-negotiable foundation of Data Science and AI.

1. **Always start with '.describe()' and visualizations.** Never trust raw data, and never trust summary statistics alone (remember Anscombe's Quartet).
2. **Understand your distributions.** Know when to use the mean vs. the median. Check for skewness before applying algorithms that assume normality.
3. **Embrace Vectorization.** If you are writing a 'for' loop in Pandas or NumPy to perform a mathematical operation, you are likely doing it wrong. Look for the vectorized alternative.
4. **Handle missing data intentionally.** Do not blindly drop rows. Investigate *why* the data is missing. Is it Missing Completely at Random (MCAR), or is the missingness itself a signal?
5. **Optimize Data Types.** Use 'category' for columns with low cardinality (e.g., gender, diagnosis) to save massive amounts of RAM. Use 'float32' instead of 'float64' if precision allows.

10 Hands-on Practice Exercises

10.1 Exercise 1: Statistical Profiling

Generate a NumPy array of 1000 random numbers drawn from a normal distribution (mean=50, std=10). Inject 20 extreme outliers (values ¿ 200). Calculate the mean, median, and standard deviation. How do the outliers affect the mean versus the median?

10.2 Exercise 2: Pandas Data Wrangling

Create a Pandas DataFrame representing 10 students with columns: 'student$_id$', '$study_hours$', '$attendance_pct$', a

Impute the missing 'study$_h ours$'$with the median$.

Create a new column 'pass$_f ail$'$where$'$final_g rade$' >= 50is'$Pass$', $else$'$Fail$'.

Use 'groupby' to find the average 'attendance$_p ct$'$for the$'$Pass$'and'$Fail$'$groups$.

10.3 Exercise 3: Correlation Matrix

Using the student DataFrame, calculate the Pearson correlation matrix for all numerical columns. Interpret the relationship between 'study$_h ours$'and'$final_g rade$'.$Does correlation imply that stud

End of Module Notes. Master these fundamentals, and the transition to Machine Learning with Scikit-Learn will be seamless.

Victor Wandera Lumumba, MSc.
Statistician — Data Analyst — ML Expert
lumumbavictor172@gmail.com • beyonddataanalytics.online